

1) En clase vimos que la base de *Quicksort* es la buena elección de un pivote para la función *Partition* y usamos un pivote elegido de forma aleatoria. Sin embargo, existen otras maneras de elegirlo.

Una de ellas es reemplazar la función *Partition* por *MedianPartition*. En la función *MedianPartition* el pivote se elige con la función *Median*(*A*, *i*, *f*), que entrega en tiempo $O(n)$ el índice en donde se ubica la mediana del arreglo *A*[*i*..*f*].

a) [1 pt.] Modifique los pseudocódigos de *Quicksort* y *Partition* de modo de adaptarlos a este nuevo escenario (*MedianPartition*) y explique porqué *MedianPartition* es $O(n)$.

b) [2 pts.] Argumente en qué casos el uso de *MedianPartition* podría mejorar el tiempo de ejecución del peor caso de *Quicksort*.

c) [3 pts.] Otra posible mejora de *QuickSort* es cambiar a *InsertionSort* para sub-secuencias pequeñas (≤ 20 elementos). Modifique el pseudocódigo de *Quicksort* para implementar esta mejora.

a)

Quicksort(*A*, *i*, *f*)

 if *i* < *f* then

***p* ← MedianPartition(*A*, *i*, *f*)** ----- Se cambia la elección del pivote

 Quicksort(*A*, *i*, *p*-1)

 Quicksort(*A*, *p*+1, *f*)

MedianPartition((*A*, *i*, *f*)

***p* ← Median A[*i*, *f*]**

m, *M* ← secuencias vacías

 Insertar *p* en *M*

 for *x* ∈ *A*[*i*, *f*] :

 if *x* < *p* : Insertar *x* en *m*

 elif *x* > *p* : Insertar *x* en *M*

A[*i*, *f*] ← Concat(*m*, *M*)

 return *i* + |*m*|

***p* ← Median A[*i*, *f*]** por enunciado es $O(n)$, el resto de *Partition* es $O(n)$. Entonces *MedianPartition* es $O(n) + O(n) = O(n)$.

b) En *Quicksort* ocurre cuando la partición produce divide el arreglo en dos partes de tamaños muy diferentes, en particular cuando el pivote es el máximo o mínimo. Al seleccionar el Pivote como la mediana se evita seleccionar los valores que dan el peor caso.

c) Se debe agregar el control

Quicksort(*A*, *i*, *f*)

***n* ← *f* - *i* + 1**

----- Se debe agregar el cambio de quicksort a

if *n* ≤ 20 then

----- Insertion sort para sub arreglos ≤ 20

InsertionSort(*A*, *i*, *f*)

else if *i* < *f* then

p ← Partition(*A*, *i*, *f*)

 Quicksort(*A*, *i*, *p*-1)

 Quicksort(*A*, *p*+1, *f*)

o ***p* ← MedianPartition(*A*, *i*, *f*)**

Árboles 2-3

- a) [1 punto] A partir del árbol 2-3 A, inicialmente vacío, inserte los siguientes elementos en el árbol A, representando gráficamente cada paso y las operaciones relevantes para mantener la propiedad de árbol 2-3 luego de cada inserción: X, D, Y, R, W, M, C, G.

- Insertar X: Árbol vacío, se inserta un nodo 2 como raíz del árbol
 - o [X]
- Insertar D: Hay espacio en el nodo hoja (raíz), pasa a nodo 3, llaves ordenadas
 - o [D|X]
- Insertar Y: El nodo colapsa -> Split, llave central se lleva al nivel superior, aumento de altura
 - o [X]
 - o [D] [Y]
- Insertar R: Nodo hoja tiene espacio, pasa a nodo 3, llaves ordenadas
 - o [X]
 - o [D|R] [Y]
- Insertar W: Nodo [D|R] colapsa -> Split, llave central al nivel superior, hay espacio, pasa a nodo 3, llaves ordenadas
 - o [R|X]
 - o [D] [W] [Y]
- Insertar M: Nodo hoja tiene espacio, pasa a nodo 3, llaves ordenadas
 - o [R|X]
 - o [D|M] [W] [Y]
- Insertar C: Nodo [D|M] colapsa -> Split, llave central al nivel superior, nodo [R|X] colapsa, llave central al nivel superior, aumento de altura.
 - o [R]
 - o [D] [X]
 - o [C] [M] [W] [Y]
- Insertar G: Nodo hoja tiene espacio, pasa a nodo 3, llaves ordenadas
 - o [R]
 - o [D] [X]
 - o [C] [G|M] [W] [Y]

Puntaje: 0.3 por insertar en hojas, 0.3 por Split correcto, 0.4 por aumento de altura correcto

- b) [2 puntos] Proponga el pseudo código para un algoritmo de búsqueda en un árbol 2-3. El algoritmo recibe como parámetros un puntero al árbol donde buscar, un puntero al padre del árbol dónde buscar, y la llave a buscar. El algoritmo debe retornar punteros al nodo encontrado (vacío si no se encuentra) y a su padre.

```
Search2-3(A, p, k):
1  if A=∅
2    return (A, p)                /* not found */
3  if A.tipoNodo=2 :               /* nodo 2, la llave está a la izquierda */
4    if A.keyLeft=k :              /* found en un nodo 2 */
5      return (A, p)
6    if k < A.keyLeft :            /* bajamos por la izquierda */
7      return Search2-3(A.left, A, k)
8    return Search2-3(A.right, A, k) /* bajamos por la derecha */
9  if A.tipoNodo=3 :              /* nodo 3, las llave están ordenadas */
```

```

10  if A.keyLeft=k V A.keyRight=k : /* found en un nodo 3, es una de las dos llaves */
11      return (A, p)
12  if k < A.keyLeft :                /* bajamos por la izquierda */
13      return Search2-3(A.left, A, k)
14  if k > A.keyRight :               /* bajamos por la derecha */
15      return Search2-3(A.left, A, k)
16  return Search2-3(A.center, A, k) /* bajamos por el centro */

```

Puntaje: 0.5 por condiciones de término, 0.5 por diferenciar nodos tipo 2 y 3 en comparación y match, 1.0 por llamadas recursivas correctas en casos nodos 2 y 3.

- c) [3 puntos] Proponga el pseudo código para un algoritmo de inserción en un árbol 2-3. El algoritmo recibe como parámetros un puntero al árbol donde insertar, un puntero al padre del árbol dónde insertar, y la llave a insertar. Nota: Debe detallar todas las funciones necesarias para la inserción, por ejemplo Split de nodos y aumento de altura del árbol.

```

Insert2-3(A, p, k):
1  if p=∅ :                               /* el árbol esta vacío, nuevo nodo 2 */
2      makeNodo2(A,k)                     /* crea un nuevo nodo 2 (hoja) y asigna la llave */
3      return
4  (B, p) <- Search2-3(A, ∅, k)           /* buscamos, solo insertamos si no está */
5  if B=∅                                 /* not found, el padre es la hoja donde insertar */
6      if p.tipoNodo=2 :                  /* el padre es hoja nodo 2, tiene espacio */
7          makeNodo3(p,k)                 /* Cambia el tipo de nodo hoja a 3 y ordena las llaves */
8          return
9                                          /* else p es un nodo 3, al insertar k colapsa */
10     A <- Split(A, p, k)                 /* promueve la llave central de p al nivel superior en A */
11     return

Split(A, p, k)                             /* A es la raíz, p el nodo que colapsa, k nueva llave */
.   gp <- p.padre                          /* gp es el padre del nodo a hacer Split */
.   centerKey = sortKeys(p, k)             /* ajusta las llaves en p, retornando la central */
.   (pLeft, pRight) <- 3to2(p, centerKey) /* cambia p de nodo 3 a dos nodos tipo 2 */
.   if gp=∅ :                             /* estoy en la raíz, aumento de altura */
.       makeNodo2(A, centerKey)           /* crea un nuevo nodo 2 (hoja) y asigna la llave */
.       linkP(A, pLeft, pRight)          /* une la raíz con los nuevos nodos 2 */
.       return A
.   if gp.tipoNodo=2 :                    /* el nodo superior tiene espacio, llave central sube */
.       makeNodo3(gp, centerKey)          /* cambia el tipo de nodo a 3 y ordena las llaves */
.       linkP(gp, pLeft, pRight)          /* une gp (tipo 3) con los nuevos nodos 2 */
.       return A
.   if gp.tipoNodo=3 :                    /* el padre del nodo actual va a requerir Split
.       A <- Split(A, gp, centerKey)
.       linkP(gp, pLeft, pRight)          /* une gp (tipo 2 o 3) con los nuevos nodos 2 */
.       return A

```

3to2(p, k): Interesa que se entienda que los nodos son un ABB a cada lado de un nodo con llave k
makeNodo2(), makeNodo3(): Interesa que crean o modifican un nodo ajustando llaves
linkP(): Interesa que enlazan los hijos de un nodo 2 o 3 consistentemente.

Puntaje: 1.0 por Insert en árbol vacío, en nodo 2 y Split para nodo 3. 1.5 por Split que considera aumento de altura, término por nodo superior con espacio y llamado recursivo hacia la raíz. 0.5 por claridad en las demás funciones de apoyo (no necesariamente detalladas).

Al analizar una solución para ordenar un conjunto de datos por dos criterios (llave primaria “apellido” y llave secundaria “nombre”), de modo que el resultado este ordenado por la llave primaria, y para igual llave primaria, por llave secundaria, se propone el siguiente algoritmo:

```
Ordena(A, n)
    QuickSortPri(A, 0, n-1) // Ordena el arreglo A, completo,
                             // por llave primaria
    i ← 0
    mientras i ≤ n - 2
        j ← i + 1
        primaria ← A[i].p
        mientras j ≤ n - 1 y A[j].p == primaria
            j ← j + 1
        QuickSortSec(A, i, j - 1) // Ordena A, entre i y j - 1,
                                   // por llave secundaria
    i ← j
```

- a) [2 puntos] ¿Es correcto el algoritmo propuesto? Justifique.

Para correctitud: Termina y realiza lo esperado.

(1 punto) Termina: QuickSort() es correcto, luego termina. El arreglo A es de tamaño definido n, la iteración (mientras) lo recorre incrementando i y j, por tanto es finita, luego Ordena() termina.

(1 punto) Función: QuickSort() es correcto, luego el arreglo se ordena por llave primaria (apellido). La iteración posiciona los índices i y j de modo de indicar la posición de inicio y término de apellidos iguales (consecutivos por el orden anterior), e invoca a QuickSort() por nombre para aquellos registros con apellidos iguales, por lo que el arreglo queda ordenado según lo solicitado.

- b) [2 puntos] ¿Cuál es la complejidad de mejor caso y la complejidad de peor caso del algoritmo propuesto?

(1 punto) El mejor caso de QuickSort() es $O(n \log n)$ lo cual se cumple para la primera llamada, si los apellidos son todos distintos la iteración aporta un $O(n)$ sin llamados al segundo QuickSort() con lo que en el mejor caso Ordena es $O(n \log n)$.

(1 punto) Para el peor caso tenemos que QuickSort() es $O(n^2)$ (por ejemplo cuando son todos los apellidos iguales), siendo todos iguales la iteración aporta $O(n)$ y una llamada a QuickSort() que en su peor caso es $O(n^2)$, como están en suma, el peor caso de Ordena() es $O(n^2)$.

- c) [2 puntos] Considere que existe el algoritmo de ordenamiento BlockSort() el cual es Estable, Correcto, In-place, con complejidad $O(n)$, $O(n \log n)$, $O(n \log n)$ en mejor, promedio y peor caso respectivamente. Dado lo anterior, escriba una modificación de

Ordena(A,n) que se beneficie de estas características (en particular su Estabilidad), asumiendo que Ud. dispone de funciones BlockSortPri(A, i, j) y BlockSortSec(A, i, j) que ordenan el arreglo A por llave primaria o secundaria respectivamente. Argumente brevemente de porqué su versión es correcta y cómo se compara en complejidad con la versión original.

(1 punto)

```
Ordena(A, n):  
    BlockSortSec(A, 0, n-1)  
    BlockSortPri(A, 0, n-1)
```

(0.3 puntos) Correctitud: Termina, dado que BlockSort() es correcto y termina.

(0.3 puntos) Efectividad: Al ser BlockSort() estable, el problema se comporta como un problema de Orden Lineal, luego de forma similar a RadixSort() la llave secundaria es el dígito menos significativo, la primaria el más significativo, se ordena primero por nombre, y al ordenar de forma estable por apellido, el orden de los nombres se mantiene y se logra el resultado esperado.

(0.4 puntos) Complejidad: BlockSort() es $O(n)$, $O(n \log n)$, $O(n \log n)$ en mejor, promedio y peor caso respectivamente. Luego Ordena() al ser llamadas consecutivas a BlockSort() tendrá complejidad en peor caso de $O(n \log n)$, lo que es mejor que la implementación anterior.

4) Árboles binarios de búsqueda

- a) Dado un **ABB** (no necesariamente balanceado), una forma —distinta de la estudiada en clases— de eliminar una clave cuyo nodo tiene ambos hijos es la siguiente: el nodo es ocupado por su (hijo y) subárbol izquierdo, T_i , mientras que su subárbol derecho, T_d , se convierte en el subárbol derecho del nodo más a la derecha de T_i .
- i) [1 pt.] Justifica que esta eliminación respeta las propiedades de ABB.
- ii) [0.5 pts.] Muestra con ejemplos que esta eliminación puede tanto aumentar como reducir la altura del árbol original (dos ejemplos, uno para cada caso).
- b) [1.5 pts.] Considerando que un nodo por sí solo —es decir, un árbol que sólo tiene la raíz— tiene **profundidad** 0, dibuja **árboles AVL** de profundidades 2 y 4 tales que *tengan el mínimo número de nodos posible*. Recuerda que un árbol AVL con el mínimo número de nodos está compuesto por una raíz y por dos subárboles AVL, cada uno a su vez con el mínimo número de nodos; comprueba que tus árboles cumplen esta propiedad.
- c) Dados dos árboles AVL, T_1 y T_2 , y una clave k , tales que todas las claves de T_1 son menores que k y todas las claves de T_2 son mayores que k , describe un algoritmo *eficiente* para implementar la unión de T_1 , k y T_2 en un nuevo árbol AVL, T ; es decir, T tiene exactamente todas las claves de T_1 más todas las claves de T_2 y más la clave k . Por “eficiente” entendemos lo siguiente: si el número total de claves involucradas es n , entonces tu algoritmo debe ser significativamente mejor que $O(n \log n)$.
- i) [1 pt.] Considera primero el caso en que las alturas de T_1 y T_2 difieren a lo más en 1; ¿cuál es el algoritmo y cuál es su complejidad en este caso?
- ii) [2 pts.] Considera ahora el caso en que la altura de T_1 es dos o más que la altura de T_2 ; ¿cuál es el algoritmo y cuál es su complejidad en este caso?

Mi pauta

- a) i) A partir de la regla para insertar claves, que, a su vez, cumple la propiedad fundamental de ABB, sabemos que en el proceso de inserción de cualquiera de las claves que están en T_i o T_d —llamemos k a una clave cualquiera en T_i o T_d — pasamos por la clave —llamémosla j — que estamos eliminando, y que, por lo tanto, k podría haber ido a parar al lugar de j , posición en la que habría cumplido la propiedad de ABB con respecto al resto del árbol. Por lo tanto, poner el subárbol T_i en la posición que ocupaba el nodo con clave j es válido.

La pregunta entonces es, ¿qué hacemos con T_d ? De nuevo, por la propiedad de ABB, las claves de T_d son todas mayores que las claves de T_i . La única posición que corresponde a claves mayores que todas las claves de T_i , pero al mismo tiempo menores que las otras claves del árbol mayores que j es como hijo derecho de la clave más a la derecha de T_i —llamémosla m ; obviamente, esta posición está "desocupada": m sólo puede ser la clave más a la derecha de T_i si no tiene hijo derecho.

- a) ii) Para simplificar (y generalizar un poco), eliminamos la raíz del árbol. Entonces, T_i "sube" a esta posición y agregamos T_d como hijo derecho del nodo más a la derecha de T_i .

La altura del árbol original, T , era $H(T) = \max\{H(T_i), H(T_d)\} + 1$. La altura del nuevo árbol, T' , puede ser desde $H(T') = H(T_i)$ hasta $H(T') = H(T_i) + H(T_d)$, dependiendo de la profundidad del nodo más a la derecha de T_i . En el primer caso, $H(T')$ es claramente menor que $H(T)$; en el segundo, $H(T')$ claramente puede ser mayor que $H(T)$.

- b) Lo primero que hay que precisar es lo siguiente. Respecto de la frase

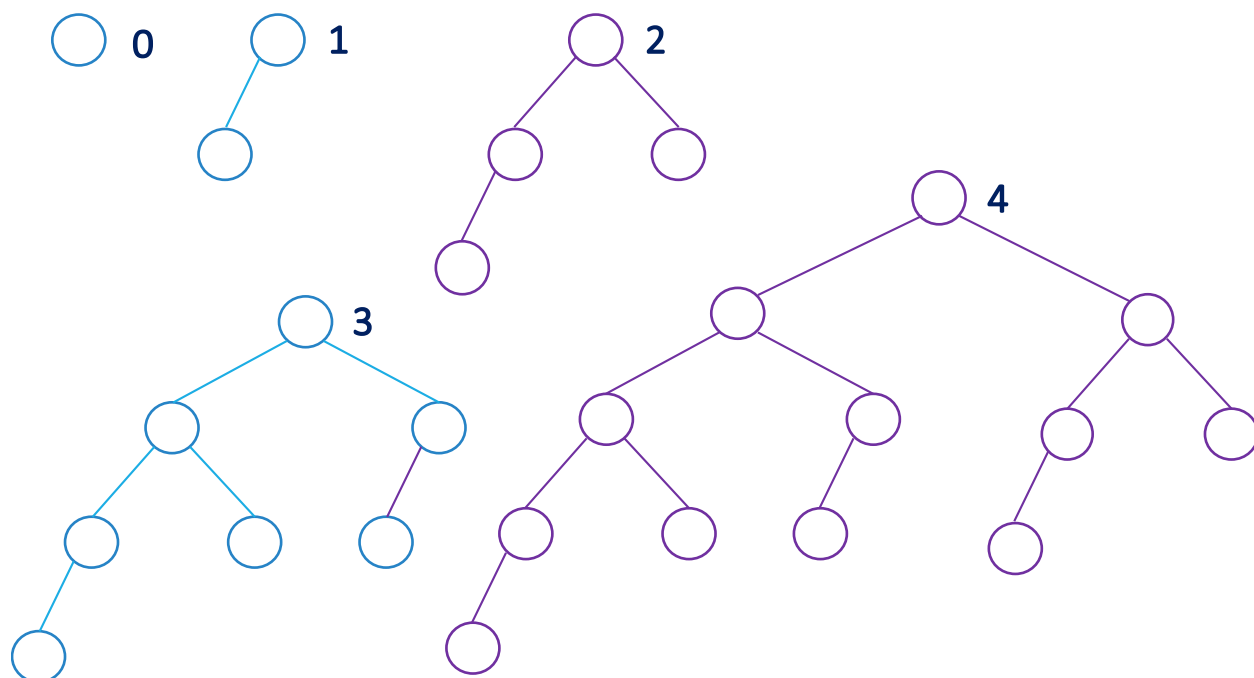
"Recuerda que un árbol AVL con el mínimo número de nodos está compuesto por una raíz y por dos subárboles AVL, cada uno a su vez con el mínimo número de nodos..."

... si el árbol AVL tuviera profundidad D , entonces, para que su número de nodos sea el mínimo posible, los dos subárboles AVL mencionados tienen que tener profundidades $D - 1$ y $D - 2$ (además del menor número posible de nodos). Es decir, el árbol pedido de profundidad 2 está compuesto por un subárbol de profundidad 1 y otro de profundidad 0; y el árbol de profundidad 4 está compuesto por un subárbol de profundidad 3 y otro de profundidad 2.

Así, si un nodo por sí solo es un árbol de profundidad 0, entonces un árbol de profundidad 1 *con el mínimo número de nodos* es un nodo con un único hijo (dos nodos en total); y **el árbol de profundidad 2 pedido** es un nodo (una raíz) con dos hijos: uno de profundidad 1 (dos nodos) y el otro de profundidad 0 (un solo nodo), como los recién descritos.

Ahora podemos construir un árbol de profundidad 3 *con el mínimo número de nodos*, consistente en un nodo (raíz) con dos hijos: el árbol de profundidad 2, del párrafo anterior, y el árbol de profundidad 1, también del párrafo anterior.

Finalmente, **el árbol de profundidad 4 pedido** se construye como un nodo (raíz) con dos hijos: el árbol de profundidad 3 y el árbol de profundidad 2.



En la fig., los números representan las profundidades de los árboles correspondientes. Los árboles pedidos son los de profundidad 2, con 4 nodos, y profundidad 4, con 12 nodos; el número de nodos es el mínimo en cada caso.

- c) i) Unimos T_1 , k y T_2 de la siguiente manera: La clave k es la clave de un nuevo nodo que es la raíz del árbol pedido, cuyo hijo izquierdo es T_1 y cuyo hijo derecho es T_2 . Ya que las claves de T_1 son menores que k y las claves de T_2 son mayores que k , este nuevo árbol cumple la propiedad de ABB; además, como las alturas de T_1 y T_2 difieren a lo más en 1, el árbol cumple también la propiedad de balance AVL. Toda esta construcción se realiza en tiempo $O(1)$, ya que no depende de cuántos nodos hay en T_1 o T_2 .
- c) ii) Partiendo de la raíz de T_1 , bajamos siempre hacia la derecha, hasta que llegamos a un nodo c tal que el subárbol con raíz c tenga la misma profundidad que T_2 . Reemplazamos c por un nuevo nodo con clave k , y cuyo hijo izquierdo es c y cuyo hijo derecho es T_2 . Como la profundidad del (nuevo) subárbol con raíz k es uno más que la profundidad del subárbol original con raíz c , es posible que uno de los ancestros del nuevo subárbol se haya desbalanceado (desde el punto de vista de balance AVL). Este desbalance se corrige ya sea con una rotación doble, si el nodo desbalanceado es el padre de k , o una rotación simple, si el desbalance se produjo más arriba. Toda esta construcción se realiza en tiempo proporcional a la diferencia de alturas entre T_1 y T_2 originales.